

- Adam Santoro, David Raposo, David GT Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia, and Timothy Lillicrap. A simple neural network module for relational reasoning. In *NeurIPS*, 2017.
- Sebastian Schuster, Ranjay Krishna, Angel Chang, Li Fei-Fei, and Christopher D Manning. Generating semantically precise scene graphs from textual descriptions for improved image retrieval. In *EMNLP Workshop*, 2015.
- Haoyue Shi, Jiayuan Mao, Tete Xiao, Yuning Jiang, and Jian Sun. Learning visually-grounded semantics from contrastive adversarial samples. In *COLING*, 2018.
- N Siddharth, T. B. Paige, J.W. Meent, A. Desmaison, N. Goodman, P. Kohli, F. Wood, and P. Torr. Learning disentangled representations with semi-supervised deep generative models. In *NeurIPS*, 2017.
- Joseph Suarez, Justin Johnson, and Fei-Fei Li. DDRprog: A clevr differentiable dynamic reasoning programmer. *arXiv:1803.11361*, 2018.
- Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *NeurIPS*, 2000.
- Ivan Vendrov, Ryan Kiros, Sanja Fidler, and Raquel Urtasun. Order-embeddings of images and language. In *ICLR*, 2016.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *MLJ*, 8(3-4):229–256, 1992.
- Jiajun Wu, Joshua B Tenenbaum, and Pushmeet Kohli. Neural scene de-rendering. In *CVPR*, 2017.
- Huijuan Xu and Kate Saenko. Ask, attend and answer: Exploring question-guided spatial attention for visual question answering. In *ECCV*, 2016.
- Jimei Yang, Scott E Reed, Ming-Hsuan Yang, and Honglak Lee. Weakly-supervised disentangling with recurrent transformations for 3d view synthesis. In *NeurIPS*, 2015.
- Zichao Yang, Xiaodong He, Jianfeng Gao, Li Deng, and Alex Smola. Stacked attention networks for image question answering. In *CVPR*, 2016.
- Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B Tenenbaum. Neural-Symbolic VQA: Disentangling reasoning from vision and language understanding. In *NeurIPS*, 2018.
- Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *CVPR*, 2015.

A CLEVR DOMAIN-SPECIFIC LANGUAGE AND IMPLEMENTATIONS

We first introduce the domain-specific language (DSL) designed for the CLEVR VQA dataset (Johnson et al., 2017a). Table 6 shows the available operations in the DSL, while Table 7 explains the type system.

Operation	Signature	Semantics
Scene	$() \rightarrow \text{ObjectSet}$	Return all objects in the scene.
Filter	$(\text{ObjectSet}, \text{ObjConcept}) \rightarrow \text{ObjectSet}$	Filter out a set of objects having the object-level concept (e.g., red) from the input object set.
Relate	$(\text{Object}, \text{RelConcept}) \rightarrow \text{ObjectSet}$	Filter out a set of objects that have the relational concept (e.g., left) with the input object.
AERelate	$(\text{Object}, \text{Attribute}) \rightarrow \text{ObjectSet}$	(Attribute-Equality Relate) Filter out a set of objects that have the same attribute value (e.g., same color) as the input object.
Intersection	$(\text{ObjectSet}, \text{ObjectSet}) \rightarrow \text{ObjectSet}$	Return the intersection of two object sets.
Union	$(\text{ObjectSet}, \text{ObjectSet}) \rightarrow \text{ObjectSet}$	Return the union of two object sets.
Query	$(\text{Object}, \text{Attribute}) \rightarrow \text{ObjConcept}$	Query the attribute (e.g., color) of the input object.
AEQuery	$(\text{Object}, \text{Object}, \text{Attribute}) \rightarrow \text{Bool}$	(Attribute-Equality Query) Query if two input objects have the same attribute value (e.g., same color).
Exist	$(\text{ObjectSet}) \rightarrow \text{Bool}$	Query if the set is empty.
Count	$(\text{ObjectSet}) \rightarrow \text{Integer}$	Query the number of objects in the input set.
CLessThan	$(\text{ObjectSet}, \text{ObjectSet}) \rightarrow \text{Bool}$	(Counting LessThan) Query if the number of objects in the first input set is less than the one of the second set.
CGreaterThan	$(\text{ObjectSet}, \text{ObjectSet}) \rightarrow \text{Bool}$	(Counting GreaterThan) Query if the number of objects in the first input set is greater than the one of the second set.
CEqual	$(\text{ObjectSet}, \text{ObjectSet}) \rightarrow \text{Bool}$	(Counting Equal) Query if the number of objects in the first input set is the same as the one of the second set.

Table 6: All operations in the domain-specific language for CLEVR VQA.

We note that some function takes Object as its input instead of ObjectSet. These functions require the uniqueness of the referral object. For example, to answer the question “What’s the color of the red object?”, there should be one and only one red object in the scene. During the program execution, the input object set will be implicitly cast to the single object (if the set is non-empty and there is only one object in the set). Such casting is named `Unique` in related works (Johnson et al., 2017b).

Type	Example	Semantics
ObjConcept	Red, Cube, etc.	Object-level concepts.
Attribute	Color, Shape, etc.	Object-level attributes.
RelConcept	Left, Front, etc.	Relational concepts.
Object	●	A single object in the scene.
ObjectSet	{●, ■}	A set of objects in the scene.
Integer	0, 1, 2, ...	A single integer.
Bool	True, False	A single boolean value.

Table 7: The type system of the domain-specific language for CLEVR VQA.

B SEMANTIC PARSING

As shown in Appendix A, a program can be viewed as a hierarchy of operations which take concepts as their parameters. Thus, NS-CL generates the hierarchies of latent programs in a sequence to tree manner (Dong & Lapata, 2016). The semantic parser adopts an encoder-decoder architecture, which contains four neural modules: (1) a bidirectional GRU encoder `IEncoder` (Cho et al., 2014) to encode an input question into a fixed-length embedding, (2) an operation decoder `OpDecoder` that determines the operation tokens, such as `Filter`, in the program based on the sentence embedding, (3) a concept decoder `ConceptDecoder` that selects concepts appeared in the input question as the parameters for certain operations (e.g., `Filter` takes an object-level concept parameter while `Query` takes an attribute), and (4) a set of output encoders $\{\text{OEncoder}_i\}$ which encode the decoded operations by `OpDecoder` and output the latent embedding for decoding the next operation. The operation decoder, the concept decoder, and the output encoders work jointly and recursively to generate the hierarchical program layout. Algorithm 1 illustrates the algorithmic outline of the semantic parser.

Algorithm 1: The String-to-Tree Semantic Parser.

```

Function parse ( $f, \{c_i\}$ ):
   $program \leftarrow \text{EmptyProgram}()$ ;
   $program.op \leftarrow \text{OpDecoder}(f)$ ;
  if  $program.op$  requires a concept parameter then
     $program.concept \leftarrow \text{ConceptDecoder}(f, \{c_i\})$ ;
  for  $i = 0, 1, \dots$  number of non-concept inputs of  $program.op$  do
     $program.input[i] \leftarrow \text{parse}(\text{OEncoder}_i(f, program.op), \{c_i\})$ ;
  return  $program$ 

```

The function `parse` takes two inputs: the current decoding state f and all concepts appeared in the question, as a set $\{c_i\}$. The parsing procedure begins with encoding the input question by `IEncoder` as f_0 , extracting the concept set $\{c_i\}$ from the input question, and invoking `parse`($f_0, \{c_i\}$).

The concept set $\{c_i\}$ is extracted using hand-coded rules. We assume that each concept (including object-level concepts, relational concepts, and attributes) is associated with a single word in the question. For example, the word “red” is associated with the object-level concept `Red`, while the word “shape” is associated with the attribute `Shape`. Informally, we call these words *concept words*. For a given question Q , the corresponding concept set $\{c_i\}$ is composed of all occurrences of the *concept words* in Q . The set of *concept words* is known for the CLEVR dataset. For natural language questions, one could run POS tagging to find all *concept words* (Andreas et al., 2016; Schuster et al., 2015). We leave the automatic discovery of concept words as a future work (Gauthier et al., 2018). We use the word embedding of the *concept words* as the representation for the concepts $\{c_i\}$. Note that, these “concept embeddings” are only for the program parsing. The visual module has separate concept embeddings for aligning object features with concepts in the visual-semantic space.